



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS
COMPONENTE: PEX1277 - MANUTENÇÃO DE SOFTWARE
DOCENTE: JOSE FERDINANDY SILVA CHAGAS

Sistema de Gerenciamento Financeiro Pessoal - FinanSee

DISCENTES:

ÂNGELA MARIA DE AQUINO SOUZA
CARLOS DANNIEL GONÇALVES DA SILVA
LEVI FILGUEIRA CHAGAS
MATEUS VICTOR DA CUNHA TOMAS
ROMULO LEVY DE CASTRO OLIVEIRA

PAU DOS FERROS – RN

2026

SUMÁRIO

1 IDENTIFICAÇÃO DA EQUIPE.....	3
2 SISTEMA WEB.....	3
2.1 Descrição do Sistema.....	3
2.2 URL do Sistema Implementado.....	3
2.3 Tecnologias Utilizadas.....	3
3 REPOSITÓRIO DO CÓDIGO FONTE.....	4
4 MECANISMO DE SOLICITAÇÃO DE MUDANÇAS.....	5
4.1 Ferramenta Utilizada:.....	5
4.2 Campos do Formulário:.....	5
4.3 Fluxo de Status das Solicitações (ciclo de vida):.....	5
5 PROTOCOLO DE MANUTENÇÃO.....	5
6 PIPELINE DE CI/CD.....	6
6.1 Código de configuração do pipeline de CI/CD:.....	6
6.2 Ferramenta de CI/CD Utilizada:.....	9
6.3 Arquivo de Configuração (caminho no repositório):.....	9
6.4 Etapas do Pipeline:.....	9
6.5 Descrição Narrativa do Pipeline.....	10
6.6 Variáveis de Ambiente.....	11
7 URL DOS SLIDES.....	12
8 CONCLUSÃO.....	13
REFERÊNCIAS.....	14

1 IDENTIFICAÇÃO DA EQUIPE

A relação dos integrantes vinculados a este projeto, acompanhada de seus respectivos números de matrícula, encontra-se detalhada na Tabela 1.

Tabela 1 - Nome dos componentes e suas matrículas

Nº	NOME COMPLETO	MATRÍCULA
1	CARLOS DANNIEL GONÇALVES DA SILVA	2022021883
2	LEVI FILGUEIRA CHAGAS	2022022050
3	MATEUS VICTOR DA CUNHA TOMAS	2022010935
4	ROMULO LEVY DE CASTRO OLIVEIRA	2022021872
5	ÂNGELA MARIA DE AQUINO SOUZA	2022010382

Fonte: Elaboração própria

2 SISTEMA WEB

2.1 Descrição do Sistema

FinanSee é uma aplicação web que foi desenvolvida pensando em auxiliar no gerenciamento financeiro e orçamentário pessoal. Em um cenário econômico frequentemente incerto, onde a busca por estabilidade financeira é constante, o FinanSee surge como uma solução eficaz e intuitiva. Nosso objetivo é fazer com que os usuários aprendam a controlar suas finanças de maneira organizada por meio de um sistema mais simples, prático e seguro, facilitando o alcance de metas financeiras e promovendo a educação financeira.

2.2 URL do Sistema Implementado

O endereço eletrônico para acessar a versão publicada do sistema desenvolvido é apresentado a seguir:

<https://manutencao-de-software-projeto-fina-vert.vercel.app/login>

2.3 Tecnologias Utilizadas

- Python 3;
- Django;

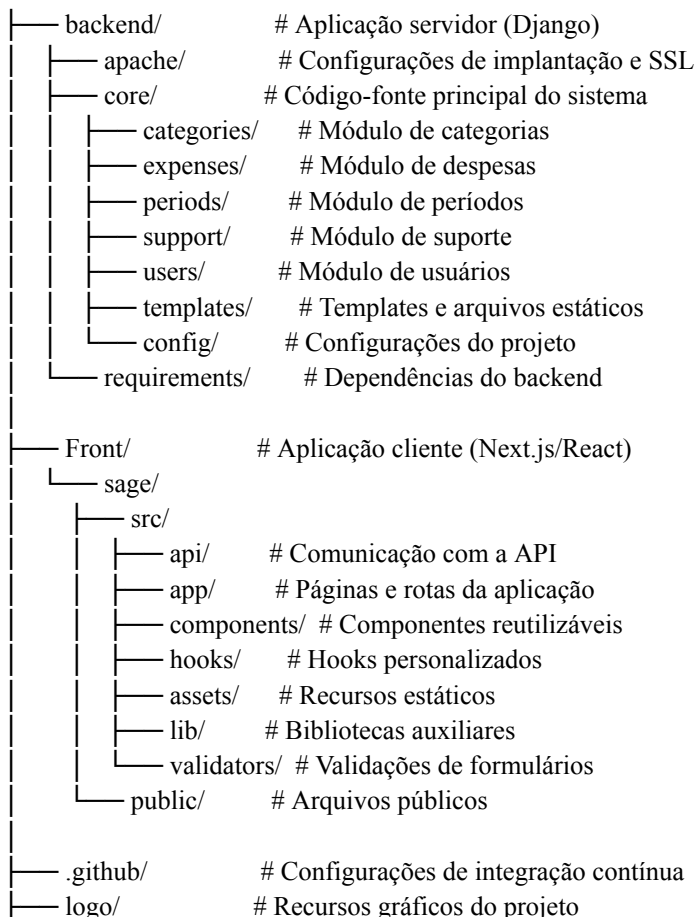
- Next.js;
- PlantUML;
- Git;
- Github;
- Docker;
- Web Token (JWT)

3 REPOSITÓRIO DO CÓDIGO FONTE

Para trabalhar com o versionamento do código do projeto foi utilizado a plataforma GitHub, na qual disponibiliza-se o acesso do projeto via URL abaixo.

https://github.com/cdaniel2002/Manutencao_de_Software_Projeto_Final.git

O repositório foi estruturado separando as responsabilidades entre o frontend e o backend da aplicação, contendo arquivos de documentação, configuração de ambiente e recursos de apoio ao processo de desenvolvimento e implantação da aplicação. Essa organização facilita a manutenção, evolução e colaboração no desenvolvimento do sistema.



— README.md	# Documentação principal
— setup.py	# Configuração de instalação
— start.bat	# Script de inicialização

4 MECANISMO DE SOLICITAÇÃO DE MUDANÇAS

4.1 Ferramenta Utilizada:

- Funcionalidade de Suporte: O sistema dispõe de uma funcionalidade dedicada ao suporte que quando acionada, redireciona o usuário para uma página onde um formulário de solicitação de alterações ou reporte de problemas, ao lado será mostrado um pequeno manual contendo exemplos de preenchimento dos campos.

4.2 Campos do Formulário:

- Título da solicitação;
- Tipo de solicitação;
- Descrição detalhada

4.3 Fluxo de Status das Solicitações (ciclo de vida):

- Aberta - Solicitação que ainda não foi analisada;
- Em análise - A solicitação está sendo analisada pela equipe de desenvolvimento para decidir como prosseguirá;
- Em andamento - A solicitação foi aprovada e analisada, sendo feita a devida correção ou manutenção do sistema;
- Fechada - Solicitação concluída após passar pelos status anteriores.

5 PROTOCOLO DE MANUTENÇÃO

As solicitações serão enviadas através de formulário e passarão por um fluxo que determinará os seus status. Após o usuário enviar o formulário o sistema informará o status em aberto, aguardando o processo de análise da solicitação do cliente. Ao entrar em análise, a equipe de desenvolvimento irá avaliar o requerimento, assim podendo decidir se é uma requisição válida ou não. O andamento da solicitação só será um processo válido para as análises que

apontarem uma requisição procedente para a equipe de desenvolvimento. Por fim, a solicitação ganha o status fechada, que indica a finalização do processo de manutenção do requerimento, além de um feedback da análise feita pela equipe.

6 PIPELINE DE CI/CD

6.1 Código de configuração do pipeline de CI/CD:

```
None
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

concurrency:
  group: ci-cd-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read
  packages: write

jobs:
  backend-tests:
    name: Backend - testes (Django)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    env:
      DJANGO_SETTINGS_MODULE: config.settings.development
      USE_POSTGRESQL: "False"
      CORS_ALLOWED_ORIGINS: "http://localhost:3000"
      BACKEND_URL: "http://localhost:8000"
      EMAIL_HOST: "smtp.example.com"
      EMAIL_HOST_USER: "Your Email"
```

```
EMAIL_HOST_PASSWORD: "changeme"
DJOSER_EMAIL_FRONTEND_PROTOCOL: "http"
DJOSER_EMAIL_FRONTEND_DOMAIN: "localhost:3000"
DJOSER_EMAIL_FRONTEND_SITE_NAME: "FinanSee"
```

steps:

- name: Checkout
uses: actions/checkout@v4
- name: Configurar Python 3.12
uses: actions/setup-python@v5
with:
 python-version: "3.12"
 cache: pip
 cache-dependency-path: backend/requirements/*.txt
- name: Instalar dependências de sistema
run: |
 sudo apt-get update
 sudo apt-get install -y --no-install-recommends \
 libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz0b \
 libgdk-pixbuf-2.0-0 libcairo2
- name: Instalar dependências Python
run: pip install -r requirements/development.txt
- name: Rodar migrações
working-directory: backend/core
run: python manage.py migrate --noinput
- name: Rodar testes
working-directory: backend/core
run: pytest -q

frontend-lint:

```
name: Frontend - lint (Next.js)
runs-on: ubuntu-latest
defaults:
  run:
    working-directory: Front/sage
```

```

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Configurar Node 20
    uses: actions/setup-node@v4
    with:
      node-version: "20"
      cache: npm
      cache-dependency-path: Front/sage/package-lock.json

  - name: Instalar dependências
    run: npm ci

  - name: Lint
    run: npm run lint

build-and-push:
  name: CD - publicar imagem no GHCR
  runs-on: ubuntu-latest
  needs: [backend-tests, frontend-lint]
  if: github.event_name == 'push' && github.ref ==
'refs/heads/main'
  env:
    IMAGE: ghcr.io/${{ github.repository_owner
}}/finanee-backend

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Configurar Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login no GHCR
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}

```



```

    password: ${ secrets.GITHUB_TOKEN }}

- name: Metadados da imagem
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${ env.IMAGE }}
    tags: |
      type=raw,value=latest
      type=sha,format=long

- name: Build e push
  uses: docker/build-push-action@v6
  with:
    context: ./backend
    file: ./backend/Dockerfile
    push: true
    tags: ${ steps.meta.outputs.tags }}
    labels: ${ steps.meta.outputs.labels }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

```

6.2 Ferramenta de CI/CD Utilizada:

Github Actions

6.3 Arquivo de Configuração (caminho no repositório):

.github/workflows/ci-cd.yml

6.4 Etapas do Pipeline:

A finalidade da Tabela 4 – Etapas do Pipeline é documentar o fluxo automatizado de integração e entrega contínua (CI/CD) do projeto.

Tabela 4 - Etapas do Pipeline

ETAPA	TRIGGER (QUANDO EXECUTA)	AÇÃO EXECUTADA	RESULTADO ESPERADO
Build	Ao enviar código para o repositório (push ou merge)	Compila o código e instala as dependências	Código pronto para testes
Testes	Após a build	Executa os testes automatizados	Garantia do funcionamento correto
Lint/Análise estática	Após os testes	Verifica padrões de código e qualidade	Código limpo conforme boas práticas
Deploy (Staging)	Após a aprovação ou merge na branch	Publica a versão em ambiente de testes	Versão disponível para validação
Deploy (Produção)	Após aprovação final	Publica a versão estável em ambiente de produção	Sistema atualizado e funcional

Fonte: Elaboração própria

6.5 Descrição Narrativa do Pipeline

O pipeline de integração e entrega contínua (CI/CD) do sistema FinanSee foi estruturado para garantir a automação das etapas de build, teste, análise e implantação. O processo é iniciado quando o desenvolvedor realiza o envio de código para o repositório, acionando automaticamente a etapa de build, responsável por compilar o projeto e instalar as dependências necessárias.

Em seguida, são executados os testes automatizados, que validam o funcionamento das principais funcionalidades e asseguram que novas alterações não causem regressões. Após os testes, ocorre a Análise estática (Lint), que verifica a qualidade e padronização do código, garantindo conformidade com as boas práticas de desenvolvimento.

Com o código validado, o pipeline segue para o deploy em ambiente de staging, onde a aplicação é publicada para homologação e validação por parte da equipe. Após a aprovação, é realizado o deploy em produção, disponibilizando a versão estável do sistema para os usuários finais.

Esse fluxo automatizado assegura maior confiabilidade, rapidez e controle nas entregas, reduzindo erros humanos e mantendo o sistema sempre atualizado e funcional.

6.6 Variáveis de Ambiente

As principais variáveis de ambiente utilizadas no projeto são:

- `DEBUG` — define se o sistema será executado em modo de desenvolvimento ou produção;
- `SECRET_KEY` — chave secreta utilizada pelo Django;
- `ALLOWED_HOSTS` — lista de domínios autorizados a acessar o backend;
- `CORS_ALLOWED_ORIGINS` — origens permitidas para requisições do frontend;
- `BACKEND_URL` — URL base do backend;
- `USE_POSTGRES` — define se o sistema utilizará PostgreSQL ou SQLite;
- `DB_NAME` — nome do banco de dados;
- `DB_USER` — usuário do banco de dados;
- `DB_PASSWORD` — senha do banco de dados;
- `DB_HOST` — host do banco de dados;
- `DB_PORT` — porta do banco de dados;
- `EMAIL_HOST` — servidor de envio de e-mails;
- `EMAIL_HOST_USER` — usuário do serviço de e-mail;
- `EMAIL_HOST_PASSWORD` — senha ou token do serviço de e-mail;
- `DJOSER_EMAIL_FRONTEND_PROTOCOL` — protocolo utilizado nos links de autenticação;

- DJOSER_EMAIL_FRONTEND_DOMAIN — domínio do frontend utilizado nos e-mails;
- DJOSER_EMAIL_FRONTEND_SITE_NAME — nome do sistema exibido nos e-mails;
- IMAGE_OWNER — dono do repositório utilizado para a imagem Docker;
- IMAGE_TAG — tag da imagem Docker publicada.

7 URL DOS SLIDES

Segue o seguinte link para ter acesso ao url do slide do projeto:

<https://www.canva.com/design/DAHM2suRrqq/Q2vVdxGH2LR0XFEezOSWGA/edit>

8 CONCLUSÃO

O desenvolvimento do sistema FinanSee consolidou os conhecimentos adquiridos na disciplina de Manutenção de Software, evidenciando a importância de um ciclo de vida robusto e estruturado. Através da implementação de um protocolo de manutenção rigoroso — abrangendo desde a triagem de solicitações até a execução e validação de mudanças — foi possível garantir a qualidade e a confiabilidade contínua da aplicação. A adoção de um pipeline de CI/CD (GitHub Actions) mostrou-se fundamental para a automação das etapas de integração, teste e entrega, minimizando falhas humanas e permitindo a aplicação eficiente de manutenções corretivas, adaptativas, evolutivas e preventivas. Conclui-se que a abordagem adotada não apenas assegura a funcionalidade e segurança do sistema de gerenciamento financeiro, mas também estabelece um modelo de trabalho sustentável, focado na manutenibilidade, na qualidade do código e na eficiência operacional a longo prazo.

REFERÊNCIAS

DJANGO SOFTWARE FOUNDATION. **Django documentation**. Disponível em: <https://www.djangoproject.com>. Acesso em: jun. 2026.

JPADILLA. **PyJWT documentation**. Disponível em: <https://pyjwt.readthedocs.io>. Acesso em: jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation**. Disponível em: <https://www.postgresql.org>. Acesso em: jun. 2026.

VERCEL. **Next.js documentation**. Disponível em: <https://nextjs.org/docs>. Acesso em: jun. 2026.

DOCKER INC. **Docker documentation**. Disponível em: <https://docs.docker.com>. Acesso em: jun. 2026.

GITHUB INC. **GitHub Actions documentation**. Disponível em: <https://docs.github.com/actions>. Acesso em: jun. 2026.